Searching Algorithms Overview

Searching means finding the **desired element inside a data structure**.

There are two major types of searching:

1. **Linear Search (Sequential Search)**
2. **Binary Search (Works only on sorted arrays)**

Linear Search

Linear Search scans the array **from start to end**.

If the current element matches the key → return the index.

When to Use

- Data is **unsorted**
- Data size is **small**

C++ Code: Linear Search

```
int linearSearch(vector<int>& arr, int key) {
    for (int i = 0; i < arr.size(); i++) {
        if (arr[i] == key) return i;
    }
    return -1;
}
```

Time Complexity

Case	Time
Best	O(1)
Average	O(n)
Worst	O(n)

2. Binary Search

Binary Search works **only on sorted arrays**.

Steps:

1. Find the middle index
2. If `arr[mid] == key` → element found
3. If `key < arr[mid]` → search in the **left half**
4. Else → search in the **right half**
5. Repeat until the element is found or the interval becomes empty

C++ Code: Binary Search

```
int binarySearch(vector<int>& arr, int key) {
```

```
    int low = 0, high = arr.size() - 1;

    while (low <= high) {
        int mid = low + (high - low) / 2;

        if (arr[mid] == key) return mid;
        else if (key < arr[mid]) high = mid - 1;
        else low = mid + 1;
    }

    return -1;
}
```

Time Complexity

Case	Time
Best	O(1)
Average	O(log n)
Worst	O(log n)

Hash Search (Hashing)

What is Hashing?

Hashing is a searching technique that uses a **hash function** to convert a key into an **index** of a hash table.

Hash table = an array where data is stored using a special formula.

Why Hashing is Used?

Hashing provides **very fast searching** because:

- No need to search entire array
- Directly access the element using index
- Average search time: **O(1)**

Components of Hashing

1. Hash Function

A mathematical function used to convert a key into an index.

Example:

```
index = key % table_size
```

2. Hash Table

Array where elements are stored.

3. Collision

When two keys produce the same index.

Example:

```
10 % 5 = 0
15 % 5 = 0  (collision)
```

Collision Handling Techniques

A. Open Addressing

1. **Linear Probing**
   Next available slot:
2. `index = (h(key) + i) % size`
3. **Quadratic Probing**
4. `index = (h(key) + i*i) % size`
5. **Double Hashing**
   Uses second hash function.

B. Closed Addressing

1. **Chaining**
   Each index has a **linked list** to store multiple elements.

C++ Code: Hashing using Chaining

```
#include <iostream>
#include <list>
using namespace std;

class HashTable {
    int size;
    list<int>* table;

public:
    HashTable(int s) {
        size = s;
        table = new list<int>[size];
    }

    int hashFunction(int key) {
        return key % size;
    }

    void insert(int key) {
        int index = hashFunction(key);
        table[index].push_back(key);
    }

    bool search(int key) {
        int index = hashFunction(key);
        for (int x : table[index]) {
            if (x == key)
                return true;
        }
        return false;
    }
};
```

```
int main() {
    HashTable h(7);

    h.insert(10);
    h.insert(20);
    h.insert(15);

    if (h.search(15))
        cout << "Found";
    else
        cout << "Not Found";

    return 0;
}
```

2. BST Search (Binary Search Tree Search)

What is a Binary Search Tree?

A **Binary Search Tree (BST)** is a binary tree where:

- **Left child < Root**
- **Right child > Root**

This property makes searching faster.

Why Use BST?

- Searching is **faster than linear search**
- Average time: **O(log n)**
- Worst case: **O(n)** (if tree becomes skewed)

Searching in BST

Searching Steps:

1. Start at root
2. If key = root → Found
3. If key < root → move to **left**
4. If key > root → move to **right**
5. Repeat until element is found or null is reached

C++ Code: BST Search

```
#include <iostream>
using namespace std;

struct Node {
    int data;
    Node* left;
    Node* right;
};

Node* createNode(int data) {
    Node* newNode = new Node();
    newNode->data = data;
```

```
    newNode->left = newNode->right = NULL;
    return newNode;
}

Node* insert(Node* root, int key) {
    if (root == NULL)
        return createNode(key);

    if (key < root->data)
        root->left = insert(root->left, key);
    else
        root->right = insert(root->right, key);

    return root;
}

bool search(Node* root, int key) {
    if (root == NULL)
        return false;

    if (root->data == key)
        return true;

    if (key < root->data)
        return search(root->left, key);
    else
        return search(root->right, key);
}

int main() {
    Node* root = NULL;

    root = insert(root, 50);
    insert(root, 30);
    insert(root, 70);
    insert(root, 60);

    if (search(root, 30))
        cout << "Found";
    else
        cout << "Not Found";
}
```

Short Notes (Exam-Friendly)

Hash Search

- Uses hash function
- Direct access using index
- Collision handling: **Chaining**, **Linear Probing**
- Time: **O(1)** average

BST Search

- Left < Root < Right
- Uses tree traversal principle
- Time: **O(log n)** average
- Worst: **O(n)**

Sorting Algorithms Overview

Sorting means **arranging data in increasing or decreasing order**.

Two main categories:

A. Comparison-Based Sorting

Compares two elements directly.
Examples:

- Bubble Sort
- Insertion Sort
- Selection Sort
- Merge Sort
- Quick Sort
- Heap Sort

B. Non-Comparison Sorting

Uses mathematical logic instead of comparing.
Examples:

- Counting Sort
- Radix Sort
- Bucket Sort

Time Complexity Comparison

Sorting Algorithm	Best	Average	Worst	Stable	In-place
Bubble Sort	O(n)	$O(n^2)$	$O(n^2)$	Yes	Yes
Insertion Sort	O(n)	$O(n^2)$	$O(n^2)$	Yes	Yes
Selection Sort	$O(n^2)$	$O(n^2)$	$O(n^2)$	No	Yes
Merge Sort	O(n log n)	O(n log n)	O(n log n)	Yes	No
Quick Sort	O(n log n)	O(n log n)	$O(n^2)$	No	Yes

Bubble Sort

Repeatedly compare adjacent elements and swap if in the wrong order.

Best For:

- Beginners
- Small datasets
- Already nearly sorted arrays

Bubble Sort Code (C++)

```
#include <bits/stdc++.h>
using namespace std;
```

```
void bubbleSort(vector<int> &arr) {
    int n = arr.size();
    for(int i = 0; i < n - 1; i++) {
        bool swapped = false;
        for(int j = 0; j < n - i - 1; j++) {
            if(arr[j] > arr[j + 1]) {
                swap(arr[j], arr[j + 1]);
                swapped = true;
            }
        }
        if(!swapped) break; // optimization
    }
}

int main() {
    vector<int> arr = {5, 3, 8, 4, 2};
    bubbleSort(arr);

    for(int x : arr) cout << x << " ";
}
```

Insertion Sort

Pick one element and insert it into its correct position in the already sorted part.

Very similar to sorting playing cards.

Best For:

- Nearly sorted data
- Small datasets
- Online data (one-by-one input)

Insertion Sort Code (C++)

```
#include <bits/stdc++.h>
using namespace std;

void insertionSort(vector<int> &arr) {
    int n = arr.size();
    for(int i = 1; i < n; i++) {
        int key = arr[i];
        int j = i - 1;

        while(j >= 0 && arr[j] > key) {
            arr[j + 1] = arr[j];
            j--;
        }
        arr[j + 1] = key;
    }
}

int main() {
    vector<int> arr = {9, 4, 7, 1, 3};
    insertionSort(arr);

    for(int x : arr) cout << x << " ";
}
```

Selection Sort

Every pass → find the **minimum element** and place it at correct position.

Best For:

- Small arrays
- When swaps are costly (fewer swaps than bubble sort)

Not Stable

(Because it swaps non-adjacent elements)

Selection Sort Code (C++)

```
#include <bits/stdc++.h>
using namespace std;

void selectionSort(vector<int> &arr) {
    int n = arr.size();

    for(int i = 0; i < n - 1; i++) {
        int minIndex = i;

        for(int j = i + 1; j < n; j++) {
            if(arr[j] < arr[minIndex])
                minIndex = j;
        }

        swap(arr[i], arr[minIndex]);
    }
}

int main() {
    vector<int> arr = {64, 25, 12, 22, 11};
    selectionSort(arr);

    for(int x : arr) cout << x << " ";
}
```

Quick Revision Table

Algorithm	Key Idea	Best Use
Bubble Sort	Swap adjacent elements	Small arrays, beginners
Insertion Sort	Insert element in sorted part	Nearly sorted or small data
Selection Sort	Pick minimum + place	When swaps need to be minimized

Merge Sort

Merge Sort is a **Divide and Conquer** sorting algorithm.

Idea:

1. Divide the array into two halves

2. Sort both halves (recursively)
3. Merge the two sorted halves

Time Complexity:

- Best = O(n log n)
- Average = O(n log n)
- Worst = O(n log n)
- Space = O(n)

Stable Sorting

Does not disturb equal elements' order.

Merge Sort Code (C++)

```
#include <bits/stdc++.h>
using namespace std;

void merge(vector<int> &arr, int left, int mid, int right) {
    int n1 = mid - left + 1;
    int n2 = right - mid;

    vector<int> L(n1), R(n2);
    for(int i = 0; i < n1; i++) L[i] = arr[left + i];
    for(int j = 0; j < n2; j++) R[j] = arr[mid + 1 + j];

    int i = 0, j = 0, k = left;

    while(i < n1 && j < n2) {
        if(L[i] <= R[j])
            arr[k++] = L[i++];
        else
            arr[k++] = R[j++];
    }

    while(i < n1) arr[k++] = L[i++];
    while(j < n2) arr[k++] = R[j++];
}

void mergeSort(vector<int> &arr, int left, int right) {
    if(left >= right) return;

    int mid = (left + right) / 2;

    mergeSort(arr, left, mid);
    mergeSort(arr, mid + 1, right);

    merge(arr, left, mid, right);
}

int main() {
    vector<int> arr = {5, 2, 8, 4, 1};
    mergeSort(arr, 0, arr.size() - 1);

    for(int x : arr) cout << x << " ";
}
```

Quick Sort

Quick Sort is a **Divide and Conquer** algorithm that uses **Partitioning**.

Idea:

1. Pick a pivot
2. Move smaller elements to left, larger to right
3. Recursively sort both sides

Time Complexity:

- Best = O(n log n)
- Average = O(n log n)
- Worst = $O(n^2)$ (when array is already sorted and pivot is last element)

In-place (no extra array)

Not Stable Sorting

Quick Sort Code (C++)

```
#include <bits/stdc++.h>
using namespace std;

int partition(vector<int> &arr, int low, int high) {
    int pivot = arr[high];
    int i = low - 1;

    for(int j = low; j < high; j++) {
        if(arr[j] < pivot) {
            i++;
            swap(arr[i], arr[j]);
        }
    }
    swap(arr[i + 1], arr[high]);
    return i + 1;
}

void quickSort(vector<int> &arr, int low, int high) {
    if(low < high) {
        int pi = partition(arr, low, high);

        quickSort(arr, low, pi - 1);
        quickSort(arr, pi + 1, high);
    }
}

int main() {
    vector<int> arr = {10, 7, 8, 9, 1, 5};
    quickSort(arr, 0, arr.size() - 1);

    for(int x : arr) cout << x << " ";
}
```